

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **2**
Topic : **2**

Graph Based Data Model

Objectives

This session will give the knowledge about

- Graph Based Data Model

Graph Based Data Model

Graph stores are **important in applications that need to analyze relationships between objects** or visit all nodes in a graph in a particular manner (graph traversal).

Graph stores are **highly optimized** to **efficiently store** graph nodes and links, and allow you to query these graphs.

Graph databases are **useful for any business problem that has complex relationships between objects** such as **social networking, rules-based engines, creating mashups**.

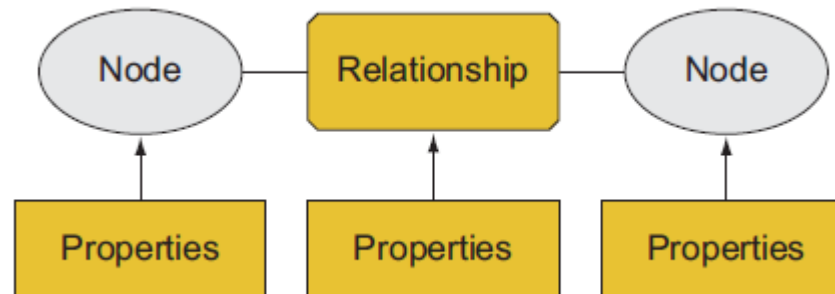
Graph systems that can quickly analyze complex network structures and find patterns within these structures.

Overview of a graph store

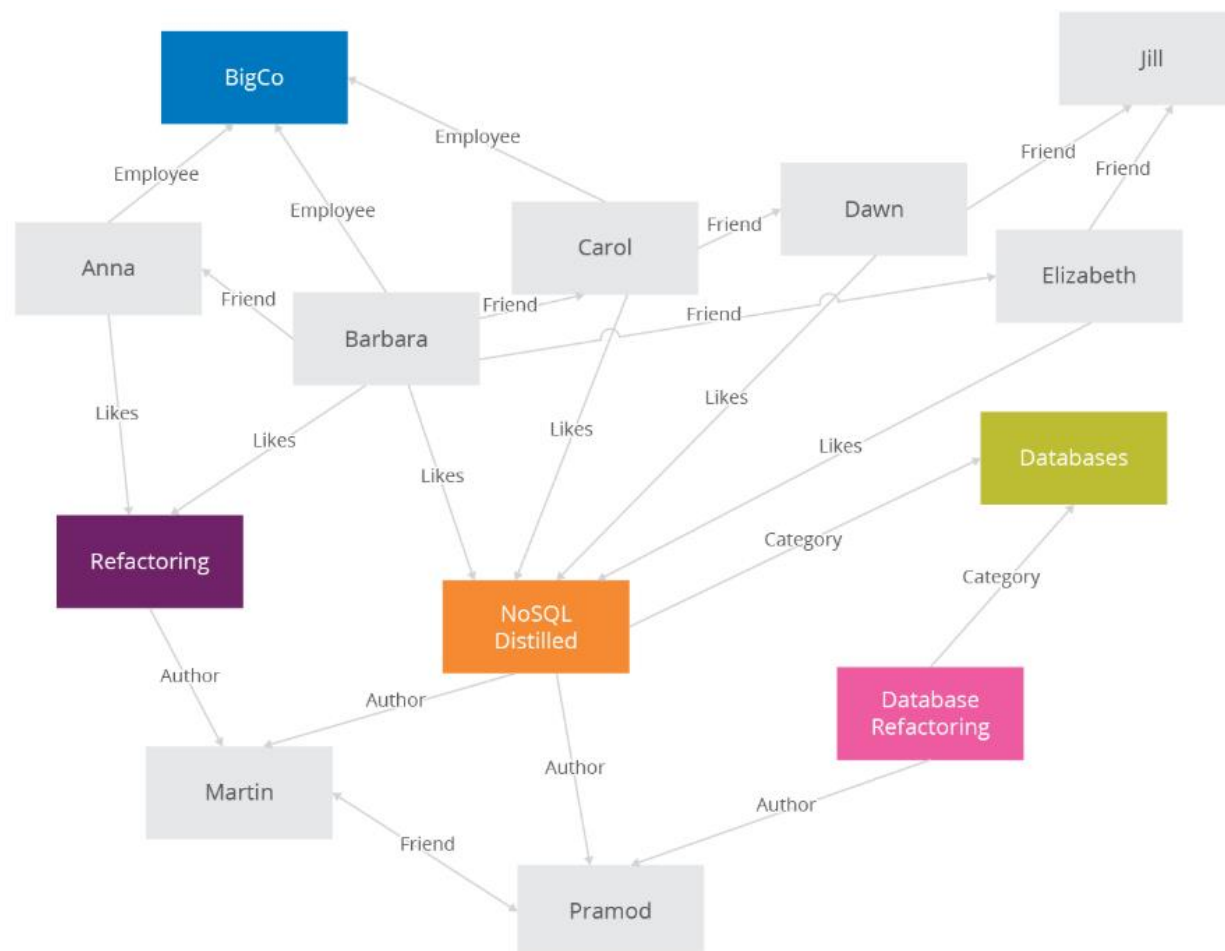
A graph store is a system that contains a sequence of nodes and relationships that are combined to create a graph.

A graph store consists of many “node-relationship-node structures”. Its also referred as triple stores.

Properties are used to describe both the nodes and relationships.



Overview of a graph store



Overview of a graph store

Graph stores are ideal when you have many items that are related to each other in complex ways and these relationships have properties (like a sister/brother of).

Graph stores allow you to do simple queries that show you the nearest neighboring nodes as well as queries that look deep into networks and quickly find patterns.

Example: Storing and retrieving of friends information in a Graph database model.

Overview of a graph store

Graph nodes are **usually representations of real-world objects like nouns**.

The **relationships can be thought of as connections between these objects** and are typically represented as arcs (lines that connect) between circles in diagrams.

Graph queries are similar to traversing nodes in a graph.

- What's the shortest path between two nodes in a graph?
- What nodes have neighboring nodes that have specific properties?
- Given any two nodes in a graph, how similar are their neighboring nodes?
- What's the average connectedness of various points on a graph with each other?

Overview of a graph store

Graph stores **relate nodes together**, understanding that two nodes with the same identifiers are the same node.

Graph stores **assign internal identifiers to nodes** and use those identifiers to join networks together.

Graph store joins are computationally lightweight and fast.

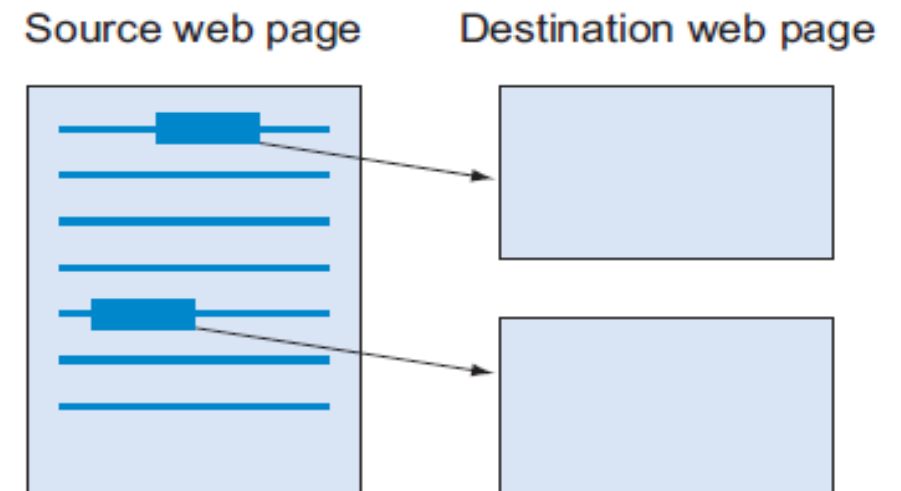
Once the graph is loaded into memory, retrieving the data doesn't require disk input and output operations.

Graph store - An example

An example of using a graph store to **represent a web page that contains links to two other web pages.**

The URL of the source web page is stored as a URL property and each link is a relationship that has a “points to” property.

Each link is represented as another node with a property that contains the destination page’s URL.



Linking external data with the RDF standard

Resource Description Format (RDF) is the W3C generalized this structure to store the information about the links between web pages and objects.

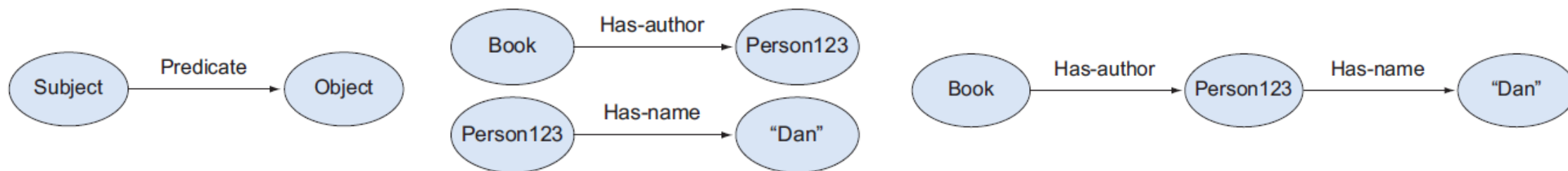
RDF was specifically **created to join together external datasets created by different organizations.**

RDF uses directed graphs, where the relationship specifically points from a source node to a destination node.

Linking external data with the RDF standard

The ability to traverse a graph relies on the fact that two nodes in different groups reference the same physical object.

For example, the Person123 node needs to globally refer to the same item. Once you determine they're the same, you can join the graphs together.



Use cases for graph stores

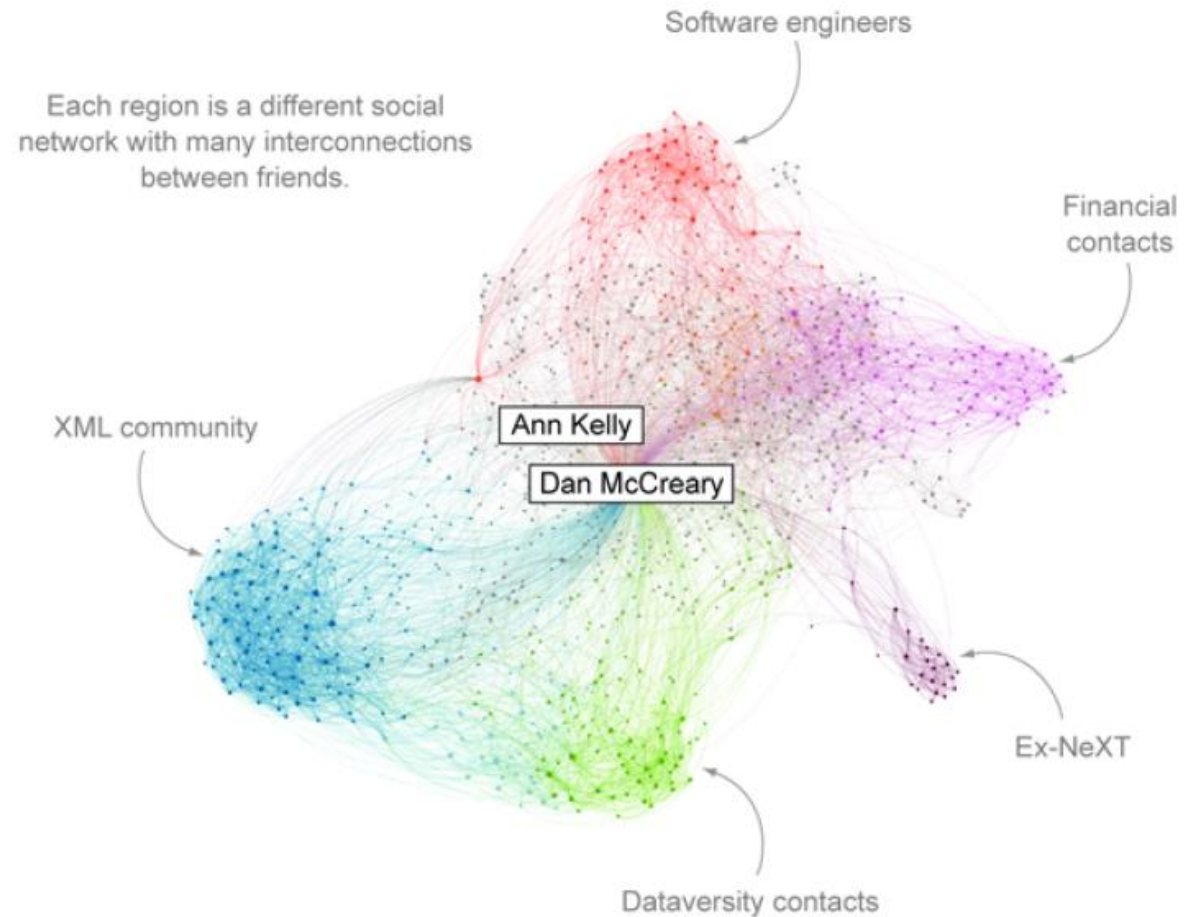
Some situations where a graph store can be used to effectively solve a particular business problem.

- **Link analysis** is used when you want to perform searches and look for patterns and relationships in situations such as social networking, telephone, or email records.
- **Rules and inference** are used when you want to run queries on complex structures such as class libraries, taxonomies, and rule-based systems.
- **Integrating linked data** is used with large amounts of open linked data to do real-time integration and build mashups without storing data.

Link Analysis

Sometimes the **best way to solve a business problem is to traverse graph data** - a good example of this is social networking.

A social network graph generated by the **LinkedIn** in Map system



Link Analysis

Graph stores are **also useful for linking data together and searching for patterns** within large collections of text documents.

Entity extraction is the process of identifying the most important items (entities) in a document.

Entities **are usually the nouns in a document like people**, dates, places, and products.

Once the key entities have been identified, they're used to perform advanced search functions.

Link Analysis

This **entity extraction process** (a type of **natural language processing or NLP**) can be combined with other tools to extract simple facts or assertions made within a document.

For example, the sentence “John Adams was born on October 19, 1735” can be broken into the following assertions:

- A person record was found with the name of John Adams and is a subject.
- The born-on relationship links the subject to the object.
- A date object record was found that has the value of October 19, 1735.

Rules, and Inference

Rules are used **to define abstract rules** that relate to an understanding of objects in a system, and how the object properties allow you to gain insight into and better use large datasets.

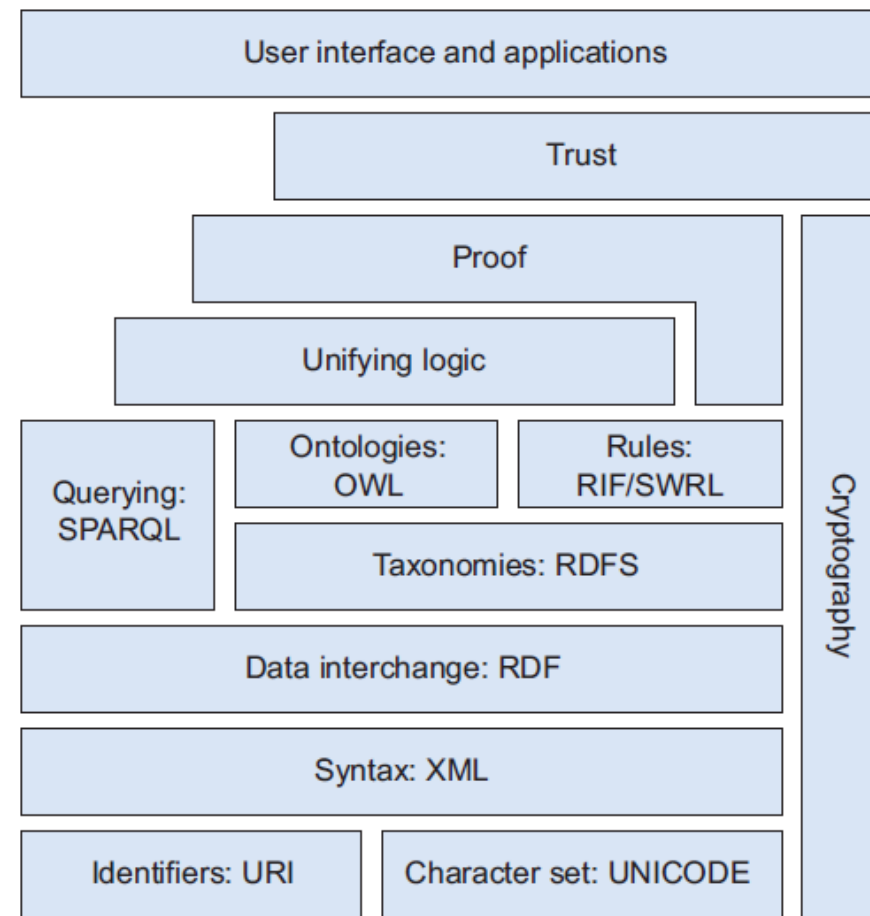
A primary use for RDF is to store logic and rules. Once you've set these rules up, you can use an inference or rules engine to discover other facts about a system.

Example: going out to dinner and you're considering two restaurants based on the reviews.

Semantic Web Stack

W3C has an entire framework of standards for using RDF to solve business problems. This framework is frequently referred to as the **Semantic Web Stack**.

- A typical semantic web stack with common low-level standards like URI, XML, and RDF at the **bottom of the stack**.
- **The middle layer** includes standards for querying (SPARQL) and standards for rules (RIF/SWRL).
- **At the top of the stack** are user interface and application layers above abstract layers of logic, proof, and trust building.



Using Graphs to Process Public Datasets

Graph stores are also useful for doing analysis on data that hasn't been created by your organization.

Example: consider, you need to do analysis with three different datasets that were created by three different organizations.

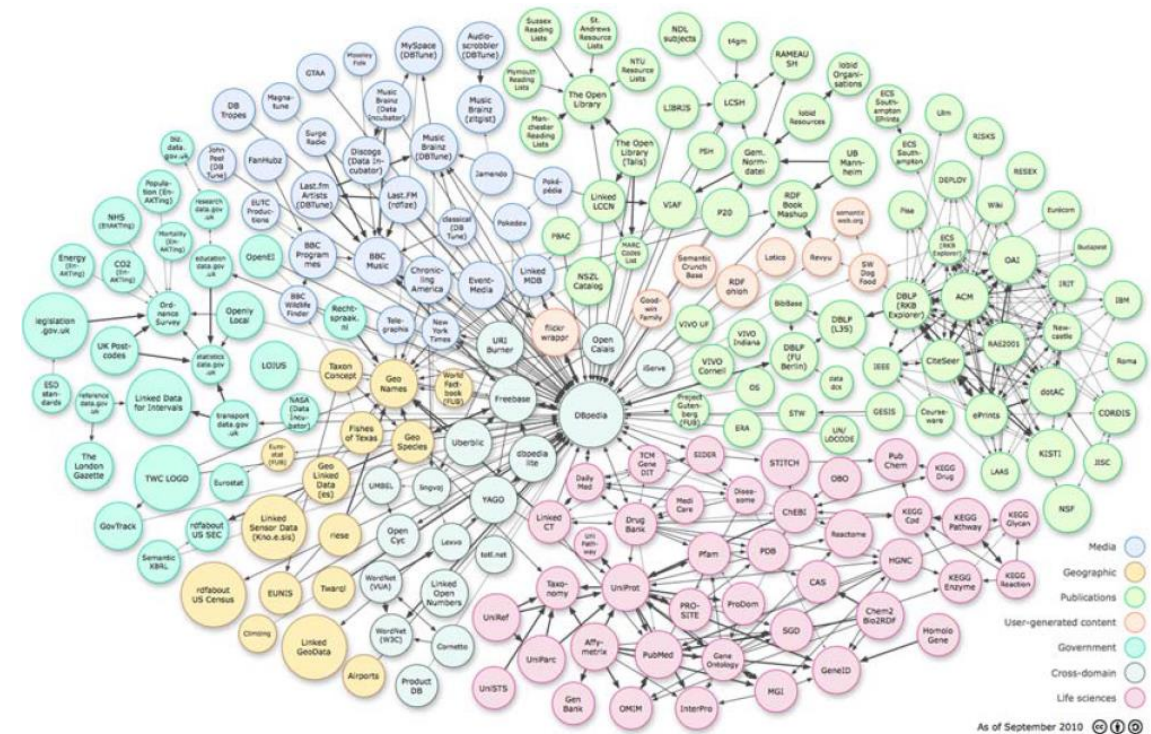
By using a set of tools referred to as **Linked Open Data** or **LOD**.

LOD strategies are important for anyone doing research or analysis using publicly available datasets.

Using Graphs to Process Public Datasets

LOD integration creates new datasets by combining information from two or more publicly available datasets that conform to the LOD structures such as RDF and URIs.

Some of the popular LOD sites called an **LOD cloud diagram**



How to create LOD Cloud data set

Find a site that has the RDF information you're looking for. This can be done in two ways

The first is to download all the RDF data on the site and load it into your graph store.

The second and more efficient method is to find a web service for the RDF site called a SPARQL endpoint.

This service allows you to submit SPARQL queries to extract the data from each of the websites you need in an RDF form that can then be joined with other RDF datasets.

How to create LOD Cloud data set

By combining the data from SPARQL queries, you can create new data mashups that join data together.

The key difference between a SPARQL query and an RDBMS is the process that creates the primary/foreign keys.

In the RDBMS, all of the keys are in the same domain.

In the LOD the data was created by different organizations, so the only way to join the data is to use consistent URIs to identify nodes.

Graph data store - Advantages

Traversing the joins or relationships is **very fast** because of the traversing persisted relationships is **faster than calculating them for every query**.

Nodes **can have different types of relationships between them**, allowing you to both represent relationships between the domain entities

Most of the value of **graph databases is derived from the relationships**. Using this property on the relationships, we **can add intelligence to the relationship**.

Adding new relationship types is easy. Changing existing nodes and their relationships is similar to data migration

Graph data store - Types

There are many graph databases available, such as

- Neo4J,
- Infinite Graph,
- OrientDB,
- FlockDB

Summary

This session will give the knowledge about

- Graph Based Data Model